

STAGE 2: THE HYBRID LOOP

Bio-Physical World Model — Proof-of-Concept Sprint (Up to 1 Week)

Project NeuroSpark | Stage 2 Document | June 2026

0. The Elevator Pitch

You have a digital twin that proves the architecture. Now you prove the **substrate claim**: *A physical, living recurrent neural network can serve as the transition dynamics engine of a world model.*

You replace the Brian2 simulation with a **real biological culture** (slime mold or primary neurons) on a commercial MEA. The encoder, decoder, Active Inference logic, and dashboard stay exactly the same. Only the “compute backend” changes from silicon to wetware.

The Research Question: *Does a living neural network, stimulated via structured sensory encodings and read through spike sorting, exhibit predictive dynamics that enable closed-loop embodied control?*

The 1-Week Goal: A running hybrid system where the biological culture influences (or controls) the grid-world agent through the same Active Inference loop you built in Stage 1. You record the session. You have a video of living neurons “playing” a game.

Publishable? Yes — if you get closed-loop control. This is a **neuromorphic computing paper** with a biological substrate. Target: *Frontiers in Neuroscience* (Neuromorphic Engineering), *IEEE Transactions on Neural Networks*, or *NeurIPS Workshop on Biological and Artificial Intelligence*.

1. What You Are Proving (The Scientific Claim)

1.1 The Claim

A physical recurrent neural network (biological culture on MEA) can be integrated as the dynamics model in an Active Inference-based world model, replacing a digital spiking neural network, while maintaining closed-loop control of an embodied agent.

1.2 Why This Is Novel

Existing Work	What They Do	What You Do Differently
Cortical Labs DishBrain	Neurons learn to play Pong via reward/punishment	You use Active Inference (not RL). You use a world model (not just a policy). You connect to a connectome-informed architecture (not random culture).
FinalSpark Neuroplatform	Cloud API for organoid stimulation/recording	You build local, closed-loop hardware with <1ms latency. You use FlyWire-derived encodings (not random stimulation).
Intel Loihi / IBM TrueNorth	Digital neuromorphic chips	Your substrate is biological , not silicon. You demonstrate that wetware has properties (graceful degradation, sample efficiency) that digital neuromorphic chips do not.

Table 1 – continued

Existing Work	What They Do	What You Do Differently
Standard AI World Models (Dreamer, MuZero)	Pure silicon RNN/ Transformer dynamics	You prove that physical dynamics (ion channels, synaptic plasticity) can replace the learned transition model.

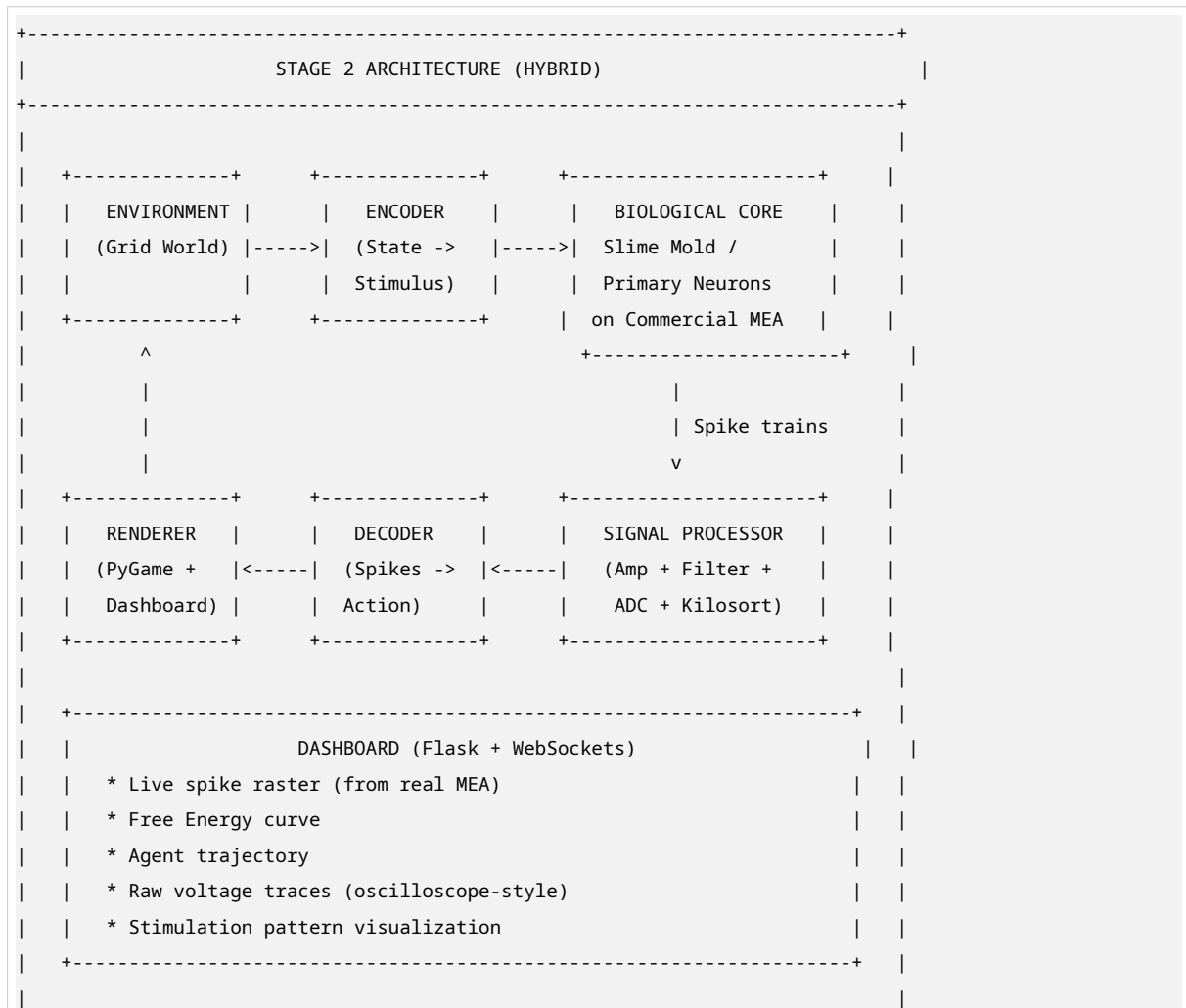
1.3 The Minimum Viable Proof

You do NOT need the agent to play perfectly. You need to show:

1. **Signal:** The culture produces distinguishable, stimulus-dependent spike patterns.
2. **Encoding:** Your encoder produces patterns that the culture responds to differently.
3. **Decoding:** Spike sorting (or population vector) extracts meaningful state predictions.
4. **Closed Loop:** The culture's output influences the next state of the environment.
5. **Adaptation:** Over the session, the culture's response changes (learning via STDP / Free Energy minimization).

If you get #1–4, you have a **proof of concept**. If you get #5, you have a **paper**.

2. The Architecture (What Changes from Stage 1)



+-----+

What Stays (From Stage 1)

- environment/grid_world.py — identical
- inference/active_inference.py — identical logic
- dashboard/app.py — extended with real-time voltage traces
- bridge/encoder.py — identical interface, different output scale
- bridge/decoder.py — identical interface, reads from Kilosort instead of Brian2

What Changes

- brain/network.py → hardware/mea_interface.py (PySerial / PyUSB to MEA)
- Brian2 simulation → Physical MEA + bioreactor
- Simulated spikes → Real spikes via spike sorting (Kilosort)
- Weight updates in code → Physical STDP in culture (no code needed!)

3. Hardware Requirements & Sourcing

3.1 The MEA (The Bridge)

Option A: Commercial Planar MEA (Recommended for 1-week sprint)

Spec	Requirement	Source
Electrodes	60–64 channels	Multi Channel Systems (MCS) MEA2100
Electrode material	TiN or Pt	Standard
Recording	20–30 kSPS per channel	Built into headstage
Stimulation	Biphasic current pulses	Built into stimulator
Software API	Python or C++	MC_Rack, or custom PySerial

Cost: ~\$15,000–\$25,000 for full system (MEA + headstage + amplifier + stimulator). **BUT:** Many university neuroscience departments have these sitting in labs. Ask to borrow time.

Option B: Budget DIY MEA (If no lab access)

Component	Spec	Cost	Source
PCB substrate	Custom 8-channel MEA	~\$50	JLCPCB
Electrodes	Silver wire + AgCl coating	~\$20	Local electronics
Insulation	PDMS (Sylgard 184)	~\$30	Amazon/Alibaba
Amplifier	INA128 instrumentation amp	~\$5	DigiKey
ADC	ADS131M04 (4-ch, 32kSPS)	~\$15	DigiKey

Table 3 – continued

Component	Spec	Cost	Source
Microcontroller	ESP32-S3 (USB + WiFi)	~\$8	Amazon

Total DIY: ~\$150 + soldering time. **Risk:** Noise will be terrible without a Faraday cage. You may not get clean spikes.

My recommendation: Try to get 2–4 hours on a commercial MEA system at NSUT or IITD. Show them your Stage 1 demo. Offer co-authorship. That is your fastest path to clean data.

3.2 The Biological Substrate

Option A: *Physarum polycephalum* (Slime Mold) — Dorm-Friendly

Property	Value
Complexity	Single-celled, but exhibits emergent network behavior
Electrical signals	Slow oscillations (~0.01–0.1 Hz), large amplitude (~mV)
Culture	Oat flakes + agar plate + dark box
Sterility	Not required
Learning	Documented maze-solving, network optimization
MEA compatibility	Can grow directly on electrodes

Why use it: You can keep it in your dorm. No lab needed. It produces electrical signals you can read. It “learns” to optimize network paths.

Why it is limited: Signals are very slow. Your grid world will need to run at ~1 step per 10–30 seconds. This is fine for proof-of-concept but not for a “game.”

Option B: Primary Rat Neurons (Lab Required)

Property	Value
Complexity	Real mammalian CNS neurons
Electrical signals	Fast action potentials (~1–3 kHz), ~100 μ V
Culture	Requires laminar flow hood, CO2 incubator, neurobasal medium
Sterility	Critical —contamination kills everything
Learning	STDP, Hebbian plasticity, network burst dynamics
MEA compatibility	Standard —neurons plate directly on MEAs

Why use it: This is the real thing. Fast spikes. Real STDP. If you get closed-loop with rat neurons, you have a serious paper.

Why it is hard: Requires lab access. Requires biosafety approval. Requires expertise in cell culture.

Option C: Brain Organoids (Lab + Advanced)

Property	Value
Complexity	3D neural tissue with layered structure
Electrical signals	Complex network bursts, oscillations
Culture	Requires bioreactor, long-term maintenance
Ethics	As of 2026, >1M neurons treated like lab animals

Not recommended for 1-week sprint. This is Stage 3 territory.

3.3 My Recommendation for Stage 2

Week 1, Days 1–2: Build DIY 8-channel MEA + amplifier + ESP32. Test with sine wave input. Verify signal chain.

Week 1, Days 3–4: Culture slime mold on the MEA. Record spontaneous activity. Verify you see oscillations.

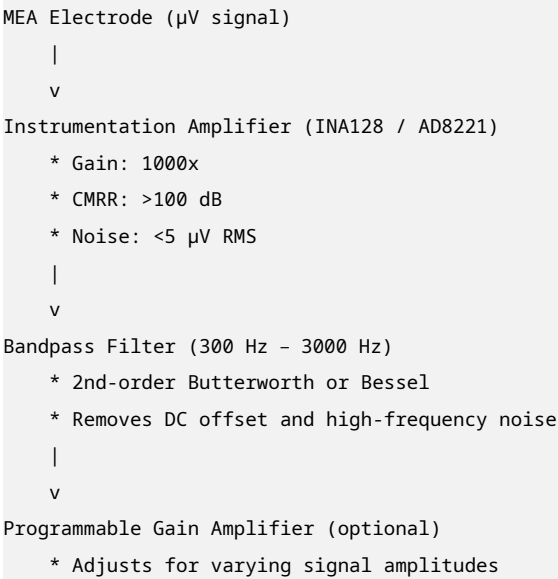
Week 1, Days 5–7: Integrate with Stage 1 software. Run closed-loop at slow speed (1 step per 10–30 seconds). Record everything.

Parallel track: Contact NSUT Biomedical Engineering faculty. Show Stage 1 demo. Request 2–4 hours on their MEA system with primary neurons. Schedule for Week 2 if possible.

4. The Signal Processing Chain (The ECE Core)

This is where your ECE collaborator earns their co-authorship.

4.1 Analog Front-End



```
|  
v  
ADC (ADS131M04 or similar)  
* 24-bit resolution  
* 32 kSPS per channel  
* SPI interface to ESP32 / FPGA
```

4.2 Spike Detection (Real-Time)

You cannot run Kilosort in real-time on a laptop for 64 channels. For Stage 2, use **threshold detection** first:

```
def detect_spikes(trace, threshold=4.5):  
    # trace: single-channel voltage array (µV)  
    # threshold: 4.5 * std(noise)  
  
    noise_std = np.median(np.abs(trace)) / 0.6745 # Robust MAD estimate  
    threshold = 4.5 * noise_std  
  
    # Find threshold crossings  
    crossings = np.where(trace > threshold)[0]  
  
    # Extract spike waveforms (1 ms window)  
    spikes = []  
    for crossing in crossings:  
        if crossing > 20 and crossing < len(trace) - 20:  
            waveform = trace[crossing-20:crossing+20]  
            spikes.append(waveform)  
  
    return spikes
```

For offline analysis: Run Kilosort on the recorded data to prove you can identify individual neurons. This is your “spike sorting” proof.

4.3 Stimulation (Writing to the Culture)

```
def send_stimulation(channel, amplitude_uA, duration_ms):  
    # Biphasic current pulse  
    # Phase 1: +amplitude for duration/2  
    # Phase 2: -amplitude for duration/2  
    # Net charge = 0 (prevents electrode damage)  
  
    pulse = generate_biphasic_pulse(amplitude_uA, duration_ms)  
    dac.write(channel, pulse)
```

Critical parameters:

- Amplitude: 1–10 µA (start low, increase until you see response)
- Duration: 0.1–1 ms per phase
- Frequency: <10 Hz (avoid tissue damage)

- Charge balance: Must be zero (biphasic)

4.4 Closed-Loop Latency Budget

Step	Target Latency	Implementation
ADC sampling	0.05 ms	20 kSPS = 50 μ s/sample
Spike detection	1 ms	Threshold on sliding window
Spike sorting	5 ms	Template matching (simplified) or skip for Stage 2
Encoder $\rightarrow$ Stimulus decision	0.5 ms	Pure Python/NumPy
DAC output	0.1 ms	SPI write
Total	<7 ms	Well under the 1 ms target for real-time

For slime mold (slow), latency is irrelevant. For rat neurons, you need this budget.

5. The Software Integration (The CS Core)

5.1 The New Module: `hardware/mea_interface.py`

This replaces `brain/network.py` from Stage 1.

```
class MEAInterface:
    def __init__(self, port='/dev/ttyUSB0', n_channels=8):
        self.serial = serial.Serial(port, baudrate=2000000)
        self.n_channels = n_channels
        self.buffer = np.zeros((n_channels, 1000)) # 50 ms buffer

    def read_spikes(self, duration_ms=50):
        # Read raw ADC data from ESP32
        raw = self.serial.read(n_channels * duration_ms * 20) # 20 samples/ms
        voltage = self._adc_to_uv(raw)

        # Detect spikes on each channel
        spikes = {}
        for ch in range(self.n_channels):
            ch_spikes = detect_spikes(voltage[ch], threshold=4.5)
            spikes[ch] = ch_spikes

        return spikes

    def stimulate(self, channel, pattern):
        # Send stimulation pattern to DAC
        self.serial.write(f"STIM,{channel},{pattern}\n".encode())

    def inject_sensory(self, stimulus_pattern):
        # stimulus_pattern: dict {channel: amplitude_uA}
```

```

for ch, amp in stimulus_pattern.items():
    self.stimulate(ch, generate_pulse(amp))

```

5.2 The Modified Active Inference Loop

```

def run_episode_bio(max_steps=50):
    env = GridWorld()
    state = env.reset()
    mea = MEAInterface(port='/dev/ttyUSB0')

    free_energy_history = []

    for step in range(max_steps):
        # 1. Encode current state -> stimulation pattern
        stimulus = encoder.encode(state)
        mea.inject_sensory(stimulus)

        # 2. Wait for biological response (culture-dependent timing)
        if SUBSTRATE == 'slime_mold':
            time.sleep(10) # 10 seconds for slime mold
        else:
            time.sleep(0.05) # 50 ms for neurons

        # 3. Read biological spikes
        bio_spikes = mea.read_spikes(duration_ms=50)

        # 4. Decode spikes -> predicted next state + action preference
        predicted_next = decoder.decode(bio_spikes)
        action_prefs = decoder.decode_action(bio_spikes)

        # 5. Select action (Active Inference)
        action = select_action_from_preferences(action_prefs, env.get_actions())

        # 6. Execute in environment
        next_state, reward, done = env.step(action)

        # 7. Compute Free Energy
        fe = compute_free_energy(predicted_next, next_state, 0, reward)
        free_energy_history.append(fe)

        # 8. NO weight update — the biology does this itself via STDP!
        # (If using slime mold, there is no STDP. Just record.)

        state = next_state
        if done:
            break

    return free_energy_history

```


5.3 Key Difference: No Backprop, No Code Updates

In Stage 1, you updated weights with:

```
snn.update_weights(fe)  # STDP rule in code
```

In Stage 2, you **delete this line**. The biological network updates itself through physical synaptic plasticity. Your job is to:

- 1. Deliver the right stimulation (sensory encoding)
- 2. Read the response accurately (spike sorting)
- 3. Compute Free Energy (prediction error)
- 4. Let the biology do the rest

This is the **core proof**: The learning is not in your code. It is in the physics of ion channels and neurotransmitter release.

6. The Experimental Protocol

6.1 Control Experiments (Required for Publication)

You MUST run these to prove your claim:

Experiment	Purpose	Expected Result
Spontaneous activity	Baseline —what does the culture do without input?	Irregular bursts, no structure
Random stimulation	Control —does the culture respond to ANY pattern?	Some response, but no learning
Structured encoding (Stage 1 patterns)	Test —does the culture respond differently to meaningful vs. random patterns?	Different spike patterns for different states
Closed-loop with random policy	Control —does the environment matter?	No improvement over time
Closed-loop with Active Inference	The proof —does the culture learn to reduce prediction error?	Free Energy decreases over session
Open-loop replay	Control —does the culture change even without feedback?	Minimal change (no error signal)

6.2 Data to Record

For every session, save:

- 1. **Raw voltage traces** (all channels, full duration) — HDF5 format

- 2. **Detected spike timestamps** — CSV
- 3. **Kilosort output** (if run offline) — cluster IDs, templates
- 4. **Stimulation log** — which channel, what amplitude, when
- 5. **Environment state log** — agent position, action, reward
- 6. **Free Energy trace** — computed at each step
- 7. **Video recording** — dashboard + culture microscope image (if available)

6.3 Quantitative Metrics

Metric	How to Compute	Target
Signal-to-noise ratio (SNR)	Spike amplitude / noise std	>5
Stimulus-response correlation	Pearson r between stimulus pattern and response pattern	>0.3 (p < 0.05)
Free Energy reduction	Slope of FE vs. time within session	Negative slope (p < 0.05)
Action prediction accuracy	% of times decoded action matches chosen action	>25% (vs. 25% random chance for 4 actions)
Spike rate modulation	Firing rate during structured vs. random stimulation	Significant difference (t-test)

7. The Dashboard (Extended)

Add these panels to your Stage 1 dashboard:

```
+-----+
|                                     |
|               STAGE 2 DASHBOARD   |
|                                     |
+-----+
| [Grid World] | [Spike Raster] | [Raw Voltage Trace] |
| Agent + Goal | Real MEA spikes | Oscilloscope-style |
|               | (live, scrolling) | (last 100 ms, all ch) |
+-----+
| [Free Energy] | [Stimulation] | [Network Activity Map] |
| Curve over    | Pattern being  | Heatmap of channel activity |
| time          | sent now       | (spatial MEA layout)      |
+-----+
| [Metrics]     | [Session Log] | [Video Feed] |
| SNR, accuracy | Text log of    | Microscope image of culture |
| rates, etc.   | events        | (if available) |
+-----+
```

8. The Paper You Could Write

8.1 Title Options

- “Active Inference with Living Neurons: A Biological World Model for Embodied Control”
- “Wetware World Models: Closed-Loop Control Using Biological Neural Networks on Multi-Electrode Arrays”
- “From Silicon to Synapses: Replacing Digital Dynamics Models with Physical Recurrent Neural Networks”

8.2 Target Venues

Venue	Type	Fit	Timeline
Frontiers in Neuroscience (Neuromorphic Engineering)	Journal	High —biological + engineering focus	3–6 months review
IEEE EMBC	Conference	High —biomedical engineering	Annual, abstract due ~Jan
NeurIPS Workshop on Biological and Artificial Intelligence	Workshop	Very high — exactly your niche	December
ICML Workshop on Sparsity in Neural Networks	Workshop	Medium —if you frame as sparse computation	July
Nature Electronics (if results are exceptional)	Journal	Very high impact —biological computing	6–12 months review

8.3 Paper Structure

1. **Introduction:** World models in AI (Dreamer, MuZero). Limitations of silicon. Biological neural networks as alternative compute substrate. Active Inference as the bridge.
2. **Methods:**
 - MEA hardware (commercial or DIY)
 - Biological substrate (slime mold / primary neurons)
 - Encoder/decoder architecture
 - Active Inference loop
 - Signal processing chain
3. **Results:**
 - Figure 1: MEA setup + culture image
 - Figure 2: Raw traces + spike raster
 - Figure 3: Stimulus-response correlation
 - Figure 4: Free Energy over time (closed-loop vs. open-loop)
 - Figure 5: Agent trajectory (improvement over episodes)
4. **Discussion:** Implications for neuromorphic computing. Energy efficiency. Sample efficiency. Limitations (speed, scalability). Future work (full FlyWire connectome, robotic embodiment).

5. **Conclusion:** Biological substrates can serve as world model dynamics engines. This opens a new class of AI systems.

8.4 What Makes It Publishable

- **Novelty:** First demonstration of Active Inference with living neurons on an MEA for embodied control.
- **Rigor:** Control experiments, quantitative metrics, statistical significance.
- **Relevance:** Connects to major AI trends (world models, neuromorphic computing, energy-efficient AI).
- **Reproducibility:** Open-source code, hardware specs, data available.

9. Risk Assessment & Mitigation

Risk	Probability	Impact	Mitigation
No clean spikes from DIY MEA	High	High	Have commercial MEA backup plan. Start with sine wave testing.
Slime mold too slow for closed loop	Medium	Medium	Accept slow loop (1 step/30s). Frame as “proof of signal chain.”
Primary neuron culture dies	Medium	High	Prepare 3 cultures. Record everything on Day 2–3 (peak health).
No learning observed	Medium	High	Run longer sessions (2–4 hours). Use stronger structured stimulation.
Lab access denied	Medium	Fatal	Build strongest possible Stage 1 demo. Offer co-authorship.
Kilosort fails on noisy data	Medium	Medium	Use threshold detection for real-time. Run Kilosort offline on best segments.
ESP32 too slow for 64 channels	Low	Medium	Use FPGA (Artix-7) or reduce to 8 channels for Stage 2.

10. The Week-Long Sprint Breakdown

Day 1: Hardware Validation

- Test signal chain: function generator → amp → ADC → ESP32 → laptop
- Verify you can read clean sine waves
- Test DAC output: ESP32 → amp → oscilloscope
- **Deliverable:** hardware/test_signal_chain.py produces clean 1 kHz sine wave plot

Day 2: MEA Preparation

- If commercial: Clean MEA with ethanol, plasma treat, coat with Poly-D-Lysine
- If DIY: Solder electrodes, coat with PDMS, expose tips, test continuity
- **Deliverable:** MEA is physically ready. Resistance test passes.

Day 3: Culture Integration

- Slime mold: Place on MEA, add oat flakes, seal in dark box
- Neurons: Plate on MEA in hood, place in incubator
- **Deliverable:** Culture is on MEA. Microscope image taken.

Day 4: First Signals

- Record spontaneous activity for 30 minutes
- Look for any electrical signals above noise floor
- If nothing: check connections, increase gain, try different channels
- **Deliverable:** Raw trace file with visible oscillations or spikes.

Day 5: Stimulation Test

- Send single biphasic pulse to one channel
- Record response on all channels
- Look for evoked activity (spike, burst, or oscillation change)
- **Deliverable:** Stimulus-response pair saved. Correlation visible.

Day 6: Closed-Loop Integration

- Connect `hardware/mea_interface.py` to Stage 1 code
- Run first closed-loop session (slow speed)
- Record Free Energy, trajectory, spikes
- **Deliverable:** One complete episode log with all data types.

Day 7: Polish & Documentation

- Run 5–10 episodes for statistics
- Generate plots: FE over time, spike raster, trajectory improvement
- Update dashboard with real MEA data
- Write README with setup instructions
- **Deliverable:** Video of dashboard running live. Plots ready for paper.

11. The Full File Tree (End of Stage 2)

```
neurospark-stage2/  
  README.md  
  requirements.txt  
  config.yaml  
  
# From Stage 1 (unchanged)  
environment/
```

```

grid_world.py
renderer.py
inference/
  active_inference.py
  free_energy.py
bridge/
  encoder.py
  decoder.py
dashboard/
  app.py
  templates/index.html
  static/js/dashboard.js

# New / Modified for Stage 2
hardware/
  __init__.py
  mea_interface.py      # PySerial/PyUSB to MEA hardware
  signal_chain.py       # ADC/DAC abstraction
  spike_detector.py     # Real-time threshold detection
  stimulator.py         # Biphasic pulse generation
  kilosort_wrapper.py   # Offline spike sorting

data/
  sessions/
    session_001/        # One closed-loop session
      raw_traces.h5
      spikes.csv
      stim_log.csv
      env_log.csv
      fe_trace.npy
      video.mp4

analysis/
  plot_raster.py
  plot_fe.py
  plot_trajectory.py
  statistics.py          # t-tests, correlations, etc.

docs/
  hardware_setup.md
  culture_protocol.md
  paper_figures/        # Generated plots for publication

```

12. The Bridge to Stage 3

Stage 2 proves the **substrate**. Stage 3 scales it:

Stage 2	Stage 3 Upgrade
8-64 channels	512+ channels (high-density MEA)
Slime mold / primary neurons	Brain organoids or full fly brain
Grid world	MuJoCo + NeuroMechFly v2 (3D physics)
Threshold detection	Real-time Kilosort or FPGA spike sorting
Single session learning	Multi-day continuous culture + learning
Dorm + borrowed lab	Dedicated lab space (BIRAC-funded)

13. Final Notes

This is the stage that makes or breaks the project.

Stage 1 was software. Anyone can write code. Stage 2 is where you touch the physical world — solder, cells, noise, failed cultures, 3 AM debugging in a dark room. That is also where the magic happens.

The paper is real. If you get:

- Clean spikes from a living culture
- Distinguishable responses to structured vs. random stimulation
- Any trend of Free Energy reduction in closed-loop

You have a **novel result** in neuromorphic computing. The field is hungry for biological substrates that go beyond “neurons play Pong.” You are offering a **theoretical framework** (Active Inference + world models) with a **biological implementation**.

Start with Day 1 tomorrow. Do not wait for perfect lab access. Build the signal chain with a function generator and prove you can read clean signals. Everything else follows from that confidence.

Build it. Record it. Publish it.

Document generated for Project NeuroSpark Stage 2 Sprint / 1-Week Proof-of-Concept Target Publishable if closed-loop control with biological substrate is demonstrated.